

RAG Öğrenci Sunumları — Hazırlık Rehberi

Naive RAG → Kurumsal RAG eğitim projesi · 7 konu · 7 sunum

Genel Beklentiler (tüm sunumlar için)

- Süre 15–20 dk + 5 dk soru. Teorik anlatım en fazla yarısı; kalanı projedeki ilgili aşama dosyası üzerinden canlı demo/deney.
- Her sunumda en az bir ölçülmüş deney: degerlendirme.py ile parametre değiştirmeden önce/sonra isabet oranı. "Daha iyi hissettirdi" değil, sayı konuşur.
- README'deki 6 analiz sorusundan kendi konunuza düşenleri mutlaka yanıtlayın (her bölümde belirtildi).
- Test korpusu (37 dosya, karayolları kurumu) ve 171 soruluk degerlendirme/sorular.csv setinden örneklerle anlatın.

1. Belge Hazırlama Süreci (Ingestion Pipeline)

İlgili dosya: asama1_belge_okuma.py — RAG'in "çöp girerse çöp çıkar" katmanı; indeks kalitesi burada belirlenir.

Kapsam

- **Format çeşitliliği:** txt, docx, pdf, xlsx okuma; her format için Python yaklaşımı (pypdf/pdfplumber, python-docx, openpyxl).
- **PDF zorlukları:** Çok sütunlu düzen, tablolar, tekrar eden üstbilgi/altbilgi; taranmış (görüntü) PDF'lerde OCR ihtiyacı (Tesseract, PaddleOCR) — projede OCR bilinçli olarak yok, nedenini açıklayın.
- **Tablo çıkarımı:** Satır-sütun yapısı düz metne çevrilirken bağlam korunmalı. "Taşlıdere Köprüsünün yük sınırı" sorusu PDF tablosundan gelir — canlı gösterin.
- **Temizlik:** Encoding, fazla boşluk, sayfa başlığı tekrarlarının temizlenmesi.
- **Belge metadata'sı:** Dosya adı, tür, sayfa numarası, xlsx'te sheet adı — sitasyonun ham maddesi burada üretilir.
- **Kurumsal boyut:** Artımlı indeksleme (değişen belgeyi yeniden işleme), erişim yetkileri, versiyonlama (bkz. gelistirme_odevleri.md).

Sunumda yanıtlanması beklenen sorular

- Taranmış bir PDF neden boş çıkar? Çözüm ne olurdu?
- Excel'de sheet adı sitasyona neden taşınmalı? ("5. Bölgede kaç kar küreme aracı var?" örneği)
- Kötü parsing retrieval'ı nasıl bozar? Bozuk bir belgeyle örnek gösterin.

2. Chunking Stratejileri ve Metadata

İlgili dosya: asama2_parcalama.py

Kapsam

- **Neden parçalıyoruz:** Embedding modelinin girdi sınırı + retrieval odağı + LLM bağlam bütçesi.
- **Chunk size / overlap:** Tipik başlangıç 256–1024 token, overlap %10–20. Yaygın framework varsayılanları: LlamaIndex 1024 token / 20 overlap; LangChain örneklerinde 1000 karakter / 200 overlap. "Doğru" değer korpusa bağlıdır — degerlendirme.py ile ölçülür.

- **Boyut ödünleşimi:** Küçük chunk isabetli ama bağlamı eksik; büyük chunk bağlam zengin ama embedding "bulanıklaşır" ve gürültü taşır.
- **Stratejiler:** Sabit boyut · recursive (paragraf→cümle→karakter) · cümle/paragraf temelli · başlık (Markdown) temelli · semantic chunking · tablo-duyarlı · parent-child (small-to-big) · late chunking.
- **Chunk metadata:** Kaynak dosya, sayfa/sheet, bölüm başlığı, chunk id. Sitasyon etiketi retrieval katmanında üretilir, LLM'e bırakılmaz.
- **Metadata filtreleme:** Tarih, birim, belge türü ile daraltma senaryoları.

Sunumda yanıtlanması beklenen sorular

- README analiz #3: Chunk boyutu büyüyünce/küçülünce retrieval nasıl değişiyor? Üç farklı boyutla (örn. 300/800/1500) ölçüp tablo gösterin.
- README analiz #5: Metadata filtreleme ne zaman gerekli olur?
- Overlap neden var? Sıfır yapınca hangi soru tipleri bozulur?

3. Embedding Modelleri

İlgili dosya: asama3_gomme.py — projede BGE-M3 (Ollama) kullanılıyor.

Kapsam

- **Temel kavramlar:** Vektör temsili, kosinüs benzerliği, normalizasyon; boyut (384–4096) ↔ depolama/hız ilişkisi.
- **Model özellikleri:** Çok dillilik, girdi uzunluğu, instruction-aware kullanım (query/passage öneki), Matryoshka (boyut kısaltma), BGE-M3'ün dense+sparse+multi-vector yapısı.
- **Açık model vs API:** Veri gizliliği, maliyet, sürüm sabitleme — kurumsalda lokal modelin tercih nedenleri. Model değişirse TÜM korpus yeniden embed edilir; indeks ve sorguda aynı model/sürüm şart.
- **Türkçe performansı:** TR-MTEB (2025) benchmark'ında multilingual-e5-large en istikrarlı; TurkEmbed / TurkEmbed4Retrieval Türkçe'ye özel güncel modeller; BGE-M3 pratik ve Ollama'da hazır olduğu için projede seçildi.

Model	Boyut / Vektör	Bağlam	Öne çıkan özellik
BGE-M3 (BAAI, açık)	568M / 1024	8192	Dense+sparse+multi-vector, 100+ dil, Ollama'da hazır
multilingual-E5-large (MS, açık)	560M / 1024	512	TR-MTEB'de en istikrarlı çok dilli model
Nomic Embed v2 (açık)	MoE, ~305M aktif	—	Ucuz/hızlı çok dilli, Matryoshka
Jina embeddings v3 (açık)	570M / 1024	8192	Late chunking, göreve özel LoRA
Qwen3-Embedding 0.6B–8B (açık)	0.6–8B / 1024–4096	32K	MTEB çok dilli lideri (açık), instruction-aware
OpenAI text-embedding-3 (API)	? / 1536–3072	8191	Kapalı; yaygın, güçlü; veri dışarı çıkar
TurkEmbed4Retrieval (açık)	305M	—	Türkçe'ye özel, MS-MARCO-TR ile eğitilmiş

Sunumda yanıtlanması beklenen sorular

- Projede neden BGE-M3 seçildi? Alternatifi ne zaman düşündünüz?
- "Kar mücadelesinde hangi araçlar kullanılıyor?" sorusu dokümanda "kar küreme ekipmanları" geçerken nasıl bulunuyor? (dense aramanın anlamsal gücü — canlı gösterin)

- Vektör boyutu büyüyünce ne kazanılır, ne kaybedilir?

4. Vektör Veritabanı

İlgili dosya: asama4_vektor_deposu.py — projede basit kalıcı depo; gerçek DB'lere geçiş gelistirme_odevleri.md'de.

Kapsam

- **Neden gerekli:** Brute-force arama $O(n)$ — milyonlarca vektörde çöker. ANN indeksleri (HNSW, IVF, PQ) recall/hız dengesiyle log-benzeri arama sağlar.
- **Bir vektör DB'den beklenenler:** Kalıcılık, CRUD (belge silme/güncelleme), metadata filtreleme, hybrid arama desteği, yedekleme, erişim kontrolü.
- **Parametrik tasarım:** Depo katmanını soyutlayıp Chroma ↔ Qdrant ↔ pgvector değiştirilebilir yapmak; seçim kriterleri: veri boyutu, filtreleme ihtiyacı, mevcut altyapı, operasyon ekibi.

DB	Tür	Güçlü yanı	Ne zaman seçilir
Chroma	Gömülü (embedded)	Sıfır kurulum, Python-yerel	Prototip, küçük korpus, eğitim
Qdrant	Sunucu (Rust)	Payload filtreleme, quantization, hybrid	Üretim, orta-büyük ölçek
Milvus	Dağıtık sunucu	Milyar ölçek, GPU indeksleme	Çok büyük kurumsal korpus
pgvector	Postgres uzantısı	SQL ile join, mevcut DB altyapısı	Zaten Postgres varsa, orta ölçek
FAISS	Kütüphane (DB değil)	Çok hızlı ANN araştırma aracı	Kalıcılık/CRUD gerekmiyorsa

Sunumda yanıtlanması beklenen sorular

- HNSW'de M ve ef_search parametreleri neyi kontrol eder?
- Projedeki basit dosya tabanlı deponun sınırları neler? Hangi noktada gerçek DB şart olur?
- Ne zaman pgvector yeterlidir, ne zaman Qdrant/Milvus gerekir?

5. Retrieval Pipeline

İlgili dosya: asama5_getirme.py — retrieval'ı LLM'siz test etme alışkanlığını vurgulayın.

Kapsam

- **Dense search:** Embedding benzerliğiyle anlamsal arama; kelime eşleşmesi gerekmez.
- **Sparse / keyword search:** BM25 (tf-idf ailesi): kısaltma, ürün kodu, özel ad aramalarında vazgeçilmez (HGS, KÖSOB, WIM gibi terimler).
- **Hybrid search:** Dense + sparse sonuçlarını birleştirme; RRF (Reciprocal Rank Fusion, tipik $k=60$) veya skor normalizasyonu.
- **Top-K:** Küçük K bilgi kaçıır, büyük K gürültü ve bağlam maliyeti getirir. degerlendirme.py --k ile ölçün.
- **Distance threshold:** Alakasız sonucu kesme; "Yemekhane kaçta açılıyor?" sorusunda eşik olmadan sistem uydurmaya malzeme bulur.
- **MMR:** Sonuç çeşitliliği; neredeyse aynı chunk'ların tekrarını önler.
- **Reranker:** Cross-encoder (örn. bge-reranker-v2-m3): geniş getir (K=30-50) → yeniden sırala → en iyi 3-5'i LLM'e ver. Kalite artışı ↔ gecikme maliyeti.

- **Pipeline mimarisi:** Recall aşaması (hybrid, geniş) + precision aşaması (rerank, dar) olarak iki katmanlı mimari.

Sunumda yanıtlanması beklenen sorular

- README analiz #1: Neden sadece embedding araması yetmez? BM25 ile dense birbirini nasıl tamamlar? (es_anlamli vs tekil soru tiplerinde isabeti karşılaştırın)
- README analiz #2: Top-K kaç seçilmeli? --k 3/8/15 ile ölçüm gösterin.
- README analiz #4: Benzerlik eşiği neden var? Kaldırınca ne oluyor?
- Bi-encoder ile cross-encoder farkı nedir; reranker neden her sorguda embedding modelinden pahalı?

6. Prompting

İlgili dosya: asama6_naive_rag.py (--goster bayrağıyla LLM'e giden prompt incelenebilir).

Kapsam

- **Üç katman:** System prompt (rol + kurallar) · Context prompt (getirilen chunk'lar + sitasyon etiketleri) · kullanıcı sorusu.
- **Answer rules:** Yalnızca bağlamdaki bilgiyle yanıtla · bağlamda yoksa "bilmiyorum" de · her iddiaya kaynak göster · sorunun dilinde yanıtla · yorum/tahmin ekleme.
- **Halüsinasyon önleme:** Eşik (retrieval) + prompt kuralları birlikte çalışır; sitasyon retrieval katmanında hazırlanır çünkü LLM kaynak uydurabilir.
- **Bağlam formatı:** Chunk sınırlayıcıları, kaynak etiket formatı, chunk sırası ("lost in the middle" etkisi), bağlam penceresi bütçesi.
- **Çelişen kaynaklar:** Korpustaki gerçek KGM belgeleri kurgusal yönergelerle bilerek çelişir (tartı toleransı, KÖSOB ↔ KÖİ). Prompt çelişkiyi nasıl ele almalı: tek doğru mu seçmeli, çelişkiyi mi raporlamalı?
- **Güvenlik:** Belge içinden gelen talimat metinleri (prompt injection) riskine kısaca değinin.

Sunumda yanıtlanması beklenen sorular

- README analiz #6: Sitasyon neden retrieval katmanında hazırlanır, LLM'e neden güvenilmez?
- "Yemekhane kaçta açılıyor?" halüsinasyon testi nasıl geçiliyor? Answer rules'tan birini silip 'yok' tipi sorularla yeniden deneyin, farkı gösterin.
- Aynı soruyu kurallı ve kuralı prompt'la sorup çıktıları yan yana gösterin.

7. Lokal LLM Seçimi (Kaynaklara Göre)

İlgili dosya: asama6_naive_rag.py — projede qwen3:4b (Ollama).

Kapsam

- **Açık vs kapalı:** Kapalı modeller (GPT-5.x, Claude, Gemini) genelde daha güçlü ama API: kurumsal belge dışarı çıkar, sorgu başı maliyet, internet şartı. Açık model: gizlilik, offline, sabit maliyet, ince ayar imkânı.
- **Açık ekosistem (2026):** Qwen3/Qwen3.5 (0.6B–235B+), Llama, Gemma 4, Mistral, DeepSeek, GLM-5.2, Kimi K2.6/K3, OpenAI gpt-oss-20b/120b. Grok'un yalnız eski sürümleri açık (Grok-1, Grok 2.5); güncel Grok kapalı. Dikkat: GLM-5.2, Kimi, DeepSeek gibi liderler yüzlerce milyar parametrelili MoE — "açık kaynak ≠ lokalde çalışır".

- **RAG özelinde kriterler:** RAG'de LLM'den beklenen ham bilgi değil: bağlamı okuma, bağlama sadakat (faithfulness), talimat takibi ("bilmiyorum" diyebilme), Türkçe akıcılık. Küçük model + iyi retrieval > büyük model + kötü retrieval.
- **Quantization:** Q4 quantization kaliteden az ödün verip boyutu ~4 kat küçültür; Ollama varsayılanı genellikle Q4.

Model (Ollama)	Yaklaşık bellek (Q4)	RAG'de beklenti
qwen2.5:1.5b	~1.5-2 GB	Çok hızlı; basit tekil sorular; talimat takibi zayıf, halüsinasyon riski yüksek
qwen3:4b (projedeki)	~3-4 GB	Hız/kalite dengesi; çoğu ofis donanımında çalışır; Türkçe yeterli
qwen3:8b / 9B sınıfı	~6-8 GB	Daha iyi sadakat ve çok-belgeli akıl yürütme; GPU önerilir
gpt-oss-20b, Gemma 4 12B	~12-16 GB	Güçlü sunucu/iş istasyonu; kurumsal pilot için üst seçenek
GLM-5.2, Kimi, DeepSeek	Çok GPU / bulut	Açık ama lokal tek makinede pratik değil

Sunumda yanıtlanması beklenen sorular

- Projede neden qwen3:4b seçildi? Hangi kaynak kısıtında 1.5B'ye inmek, hangi durumda 8-9B'ye çıkmak mantıklı?
- asama6'yı iki farklı modelle çalıştırıp aynı 10 soruda yanıt kalitesi, "bilmiyorum" davranışı ve süreyi karşılaştırın (özellikle 'yok' tipi sorular).
- Lisanslar (Apache-2.0, MIT vs özel lisanslar) kurumsal kullanımda neden önemli?